

Documentacao MyVensim

AUTHOR
Versão 1.0

Sumário

Table of contents

Hierarquia de Diretórios

Diretórios

funcional

- funcional_tests.cpp
- funcional_tests.h
- main.cpp

src

- flow.cpp
- flow.h
- main.cpp
- model.cpp
- model.h
- system.cpp
- system.h

test

funcional

- funcional_tests.cpp
- funcional_tests.h
- main.cpp

unit

- main.cpp
- unit_tests.cpp
- unit_tests.h

unit

- main.cpp
- unit_tests.cpp
- unit_tests.h

Índice Hierárquico

Hierarquia de Classes

Esta lista de hierarquias está parcialmente ordenada (ordem alfabética):

- Flow

 - ExponencialFlow

 - FlowComplex

 - LogisticaFlow

- Model

- System

Índice dos Componentes

Lista de Classes

Aqui estão as classes, estruturas, uniões e interfaces e suas respectivas descrições:

ExponencialFlow

Flow

FlowComplex

LogisticaFlow

Model

System

Índice dos Arquivos

Lista de Arquivos

Esta é a lista de todos os arquivos e suas respectivas descrições:

- src/flow.cpp**
- src/flow.h**
- src/main.cpp**
- src/model.cpp**
- src/model.h**
- src/system.cpp**
- src/system.h**
- test/funcional/funcional_tests.cpp**
- test/funcional/funcional_tests.h**
- test/funcional/main.cpp**
- test/unit/main.cpp**
- test/unit/unit_tests.cpp**
- test/unit/unit_tests.h**

Documentação do Diretório

Referência do diretório test/funcional

Arquivos

- arquivo `funcional_tests.cpp`
- arquivo `funcional_tests.h`
- arquivo `main.cpp`

Referência do diretório src

Arquivos

- arquivo **flow.cpp**
- arquivo **flow.h**
- arquivo **main.cpp**
- arquivo **model.cpp**
- arquivo **model.h**
- arquivo **system.cpp**
- arquivo **system.h**

Referência do diretório test

Diretórios

diretório **funcional**diretório **unit**

Referência do diretório test/unit

Arquivos

- arquivo `main.cpp`
- arquivo `unit_tests.cpp`
- arquivo `unit_tests.h`

Classes

Referência da Classe ExponencialFlow

#include <funcional_tests.h>

Diagrama de hierarquia da classe ExponencialFlow:

IMAGE

Membros Públicos

- double **execute** () const override
- **Flow** ()
- **Flow** (const **Flow** &fl)
- **Flow** (std::string name)

Membros Públicos herdados de Flow

- **Flow** ()
- **Flow** (const **Flow** &fl)
- **Flow** (std::string name)
- virtual ~**Flow** ()
- **Flow** & **operator=** (const **Flow** &fl)
- void **setSource** (System *s)
- void **setTarget** (System *t)
- System * **getSource** () const
- System * **getTarget** () const
- std::string **getName** () const
- void **setName** (std::string name)

Outros membros herdados

Atributos Protegidos herdados de Flow

- std::string name
- System * source
- System * target

Documentação das funções

double ExponencialFlow::execute () const[inline], [override], [virtual]

Implementa **Flow** (p.).

```
19         {
20             //Pego a origem se for diferente de nulo
21             if (getSource() != nullptr) {
22                 //Pego o valor atual e multiplico do 0.01
23                 return 0.01 * getSource()->getValue();
24             }
25             return 0.0;
26     }
```

Flow::Flow ()

```
4 : name(""), source(nullptr), target(nullptr) {}
```

Flow::Flow (const Flow & fl)

```
10         {  
11     this->name = fl.name;  
12     this->source = fl.source;  
13     this->target = fl.target;  
14 }
```

Flow::Flow (std::string name)

```
7 : name(name), source(nullptr), target(nullptr) {}
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- test/funcional/funcional_tests.h

Referência da Classe Flow

```
#include <flow.h>
```

Diagrama de hierarquia da classe Flow:

IMAGE

Membros Públicos

- **Flow** ()
- **Flow** (const **Flow** &fl)
- **Flow** (std::string name)
- virtual ~**Flow** ()
- **Flow** & **operator=** (const **Flow** &fl)
- void **setSource** (**System** *s)
- void **setTarget** (**System** *t)
- **System** * **getSource** () const
- **System** * **getTarget** () const
- std::string **getName** () const
- void **setName** (std::string name)
- virtual double **execute** () const =0

Atributos Protegidos

- std::string **name**
- **System** * **source**
- **System** * **target**

Construtores e Destrutores

Flow::Flow ()

```
4 : name(""), source(nullptr), target(nullptr) {}
```

Flow::Flow (const Flow & fl)

```
10 {
11     this->name = fl.name;
12     this->source = fl.source;
13     this->target = fl.target;
14 }
```

Flow::Flow (std::string name)

```
7 : name(name), source(nullptr), target(nullptr) {}
```

Flow::~Flow () [virtual]

```
18 {}
```

Documentação das funções

virtual double Flow::execute () const[pure virtual]

Implementado por **ExponencialFlow** (*p.*), **FlowComplex** (*p.*) e **LogisticaFlow** (*p.*).

std::string Flow::getName () const

```
54                                     {
55     return name;
56 }
```

System * Flow::getSource () const

```
46                                     {
47     return source;
48 }
```

System * Flow::getTarget () const

```
50                                     {
51     return target;
52 }
```

Flow & Flow::operator= (const Flow & fl)

```
21                                     {
22     //Se o objeto for o mesmo que o atual, so retorna ele mesmo
23     if (this == &fl) {
24         return *this;
25     }
26     //Se for diferente, faço a atribuição
27     this->name = fl.name;
28     this->source = fl.source;
29     this->target = fl.target;
30     return *this;
31 }
```

void Flow::setName (std::string name)

```
42                                     {
43     name = n;
44 }
```

void Flow::setSource (System * s)

```
34                                     {
35     source = s;
36 }
```

void Flow::setTarget (System * t)

```
38                                     {
```

```
39     target = t;  
40 }
```

Atributos

std::string Flow::name [protected]

System* Flow::source [protected]

System* Flow::target [protected]

A documentação para essa classe foi gerada a partir dos seguintes arquivos:

- `src/flow.h`
- `src/flow.cpp`

Referência da Classe FlowComplex

#include <funcional_tests.h>

Diagrama de hierarquia da classe FlowComplex:

IMAGE

Membros Públicos

- double **execute** () const override
- **Flow** ()
- **Flow** (const **Flow** &fl)
- **Flow** (std::string name)

Membros Públicos herdados de Flow

- **Flow** ()
- **Flow** (const **Flow** &fl)
- **Flow** (std::string name)
- virtual ~**Flow** ()
- **Flow** & **operator=** (const **Flow** &fl)
- void **setSource** (**System** *s)
- void **setTarget** (**System** *t)
- **System** * **getSource** () const
- **System** * **getTarget** () const
- std::string **getName** () const
- void **setName** (std::string name)

Outros membros herdados

Atributos Protegidos herdados de Flow

- std::string **name**
- **System** * **source**
- **System** * **target**

Documentação das funções

double FlowComplex::execute () const [inline], [override], [virtual]

Implementa **Flow** (*p.*).

```
54                                     {
55         if (getSource() != nullptr) {
56             return 0.01 * getSource()->getValue(); // Equação base do fluxo complexo
57         }
58         return 0.0;
59     }
```

Flow::Flow ()

```
4 : name(""), source(nullptr), target(nullptr) {}
```

Flow::Flow (const Flow & fl)

```
10      {
11      this->name = fl.name;
12      this->source = fl.source;
13      this->target = fl.target;
14  }
```

Flow::Flow (std::string name)

```
7 : name(name), source(nullptr), target(nullptr) {}
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- test/funcional/funcional_tests.h

Referência da Classe LogisticaFlow

#include <funcional_tests.h>

Diagrama de hierarquia da classe LogisticaFlow:

IMAGE

Membros Públicos

- double **execute** () const override
- **Flow** ()
- **Flow** (const **Flow** &fl)
- **Flow** (std::string name)

Membros Públicos herdados de Flow

- **Flow** ()
- **Flow** (const **Flow** &fl)
- **Flow** (std::string name)
- virtual ~**Flow** ()
- **Flow** & **operator=** (const **Flow** &fl)
- void **setSource** (**System** *s)
- void **setTarget** (**System** *t)
- **System** * **getSource** () const
- **System** * **getTarget** () const
- std::string **getName** () const
- void **setName** (std::string name)

Outros membros herdados

Atributos Protegidos herdados de Flow

- std::string **name**
- **System** * **source**
- **System** * **target**

Documentação das funções

double LogisticaFlow::execute () const[inline], [override], [virtual]

Implementa **Flow** (*p.*).

```
36                                     {
37         //Pego a origem se ela nao for nula
38         if (getTarget() != nullptr) {
39             //Pego o valor atual
40             double p2 = getTarget()->getValue();
41             //Faço as contas pedidas pelo cliente
42             return 0.01 * p2 * (1.0 - (p2 / 70.0));
43         }
44         return 0.0;
45     }
```

Flow::Flow ()

```
4 : name(""), source(nullptr), target(nullptr) {}
```

Flow::Flow (const Flow & fl)

```
10         {  
11     this->name = fl.name;  
12     this->source = fl.source;  
13     this->target = fl.target;  
14 }
```

Flow::Flow (std::string name)

```
7 : name(name), source(nullptr), target(nullptr) {}
```

A documentação para essa classe foi gerada a partir do seguinte arquivo:

- test/funcional/funcional_tests.h

Referência da Classe Model

```
#include <model.h>
```

Membros Públicos

- **Model ()**
- **Model (const Model &mod)**
- **Model (std::string name)**
- **virtual ~Model ()**
- **Model & operator= (const Model &mod)**
- **void add (System *s)**
- **void add (Flow *f)**
- **void run (int t_initial, int t_end)**
- **std::string getName () const**
- **void setName (std::string name)**

Construtores e Destrutores

Model::Model ()

```
4 : name("") {}
```

Model::Model (const Model & mod)

```
10 {  
11     this->name = mod.name;  
12     this->systems = mod.systems;  
13     this->flows = mod.flows;  
14 }
```

Model::Model (std::string name)

```
7 : name(name) {}
```

Model::~~Model () [virtual]

```
19 {}
```

Documentação das funções

void Model::add (Flow * f)

```
47 {  
48     flows.push_back(f);  
49 }
```

void Model::add (System * s)

```
42         {
43     systems.push_back(s);
44 }
```

std::string Model::getName () const

```
33         {
34     return name;
35 }
```

Model & Model::operator= (const Model & mod)

```
22         {
23     if (this == &mod) {
24         return *this;
25     }
26     this->name = mod.name;
27     this->systems = mod.systems;
28     this->flows = mod.flows;
29     return *this;
30 }
```

void Model::run (int t_initial, int t_end)

```
52         {
53     //Enquanto o tempo nao acabar
54     for (int tempo = t_initial; tempo < t_end; ++tempo) {
55         //Crio um vetor de resultados
56         std::vector<double> results;
57
58         //Executo os fluxos e coloco o resultado no vetor de resultados
59         for (size_t i = 0; i < flows.size(); ++i) {
60             results.push_back(flows[i]->execute());
61         }
62
63         //Pego os sistemas de origem e destino dos fluxos
64         for (size_t i = 0; i < flows.size(); ++i) {
65             System* origem = flows[i]->getSource();
66             System* destino = flows[i]->getTarget();
67             //Se não forem nulos, atualizo os valores ao mesmo tempo para não ter
inconsistência
68             if (origem != nullptr) {
69                 origem->setValue(origem->getValue() - results[i]);
70             }
71             if (destino != nullptr) {
72                 destino->setValue(destino->getValue() + results[i]);
73             }
74         }
75     }
76 }
```

void Model::setName (std::string name)

```
37         {
38     name = n;
39 }
```

A documentação para essa classe foi gerada a partir dos seguintes arquivos:

- `src/model.h`
- `src/model.cpp`

Referência da Classe System

```
#include <system.h>
```

Membros Públicos

- **System** ()
- **System** (const std::string &name, double value)
- virtual **~System** ()
- **System** (const **System** &other)
- **System** & **operator=** (const **System** &other)
- std::string **getName** () const
- double **getValue** () const
- void **setValue** (double value)
- void **setName** (std::string name)

Construtores e Destrutores

System::System ()

```
4 : name(""), value(0.0) {}
```

System::System (const std::string & name, double value)

```
7                                     {
8     this->name = name;
9     this->value = value;
10 }
```

System::~~System () [virtual]

```
13 {}
```

System::System (const **System** & other)

```
16                                     {
17     this->name = other.name;
18     this->value = other.value;
19 }
```

Documentação das funções

std::string System::getName () const

```
32                                     {
33     return name;
34 }
```

double System::getValue () const

```
36                                     {
37     return value;
38 }
```

System & System::operator= (const System & other)

```
22                                     {
23     if (this == &other) {
24         return *this;
25     }
26     this->name = other.name;
27     this->value = other.value;
28     return *this;
29 }
```

void System::setName (std::string name)

```
44                                     {
45     this->name = n;
46 }
```

void System::setValue (double value)

```
40                                     {
41     this->value = v;
42 }
```

A documentação para essa classe foi gerada a partir dos seguintes arquivos:

- src/system.h
- src/system.cpp

Arquivos

Referência do Arquivo src/flow.cpp

```
#include "flow.h"
```

Referência do Arquivo src/flow.h

```
#include <string>
#include "system.h"
```

Componentes

class **Flow**

flow.h

Ir para a documentação desse arquivo.

```
1 #ifndef FLOW_H
2 #define FLOW_H
3
4 #include <string>
5 #include "system.h"
6
7 class Flow {
8 protected:
9     //Atributos
10     std::string name;
11     System *source;
12     System *target;
13
14 public:
15     //Construtor vazio
16     Flow();
17     //Construtor de cópia
18     Flow(const Flow &fl);
19     // Construtor com nome
20     Flow(std::string name);
21     //Destrutor
22     virtual ~Flow();
23     //Sobrecarga
24     Flow &operator=(const Flow &fl);
25
26     //Get e set de origem, destino e nome
27     void setSource(System *s);
28     void setTarget(System *t);
29     System *getSource() const;
30     System *getTarget() const;
31     std::string getName() const;
32     void setName(std::string name);
33
34     //Metodo virtual puro para que as subclasses façam
35     virtual double execute() const = 0;
36 };
37
38 #endif
```

Referência do Arquivo src/model.cpp

```
#include "model.h"
```

Referência do Arquivo src/model.h

```
#include <string>
#include <vector>
#include "system.h"
#include "flow.h"
```

Componentes

class **Model**

model.h

Ir para a documentação desse arquivo.

```
1 #ifndef MODEL_H
2 #define MODEL_H
3
4 #include <string>
5 #include <vector>
6 #include "system.h"
7 #include "flow.h"
8
9 class Model {
10 private:
11     //Atributos
12     std::string name;
13     std::vector<System*> systems;
14     std::vector<Flow*> flows;
15
16 public:
17     //Construtor vazio
18     Model();
19     //Construtor de copia
20     Model(const Model& mod);
21     // Construtor com nome
22     Model(std::string name);
23     //Destrutor
24     virtual ~Model();
25     //Sobrecarga
26     Model& operator=(const Model& mod);
27
28     // Métodos de adicionar sistemas e flow no modelo
29     void add(System* s);
30     void add(Flow* f);
31
32     //Metodo para simulação
33     void run(int t_initial, int t_end);
34
35     //Get e set de nome
36     std::string getName() const;
37     void setName(std::string name);
38 };
39
40 #endif
```

Referência do Arquivo src/system.cpp

```
#include "system.h"
```

Referência do Arquivo src/system.h

```
#include <string>
```

Componentes

```
class System
```

system.h

Ir para a documentação desse arquivo.

```
1 #ifndef SYSTEM_H
2 #define SYSTEM_H
3
4 #include <string>
5
6 class System {
7 private:
8     //Atributos
9     std::string name;
10    double value;
11
12 public:
13     //Construtor vazio
14     System();
15     //Construtor normal
16     System(const std::string& name, double value);
17     //Destrutor
18     virtual ~System();
19     //Construtor de copiar
20     System(const System& other);
21     //Sobrecarga
22     System& operator=(const System& other);
23
24     //Get e set dos atributos
25     std::string getName() const;
26     double getValue() const;
27     void setValue(double value);
28     void setName(std::string name);
29 };
30
31 #endif
```

Referência do Arquivo test/funcional/funcional_tests.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include <cassert>
#include "funcional_tests.h"
#include "../src/model.h"
#include "../src/system.h"
```

Funções

- void **exponentialFuncionalTest** ()
- void **logisticalFuncionalTest** ()
- void **complexFuncionalTest** ()

Funções

void **complexFuncionalTest** ()

```
84                                     { //Funcao para realizar testes complexos
85     std::cout << "Executando Teste Complexo (Sistema Q)..." << std::endl;
86
87     //Criação dos 5 sistemas soltos na memória
88     System* q1 = new System("Q1", 100.0);
89     System* q2 = new System("Q2", 0.0);
90     System* q3 = new System("Q3", 100.0);
91     System* q4 = new System("Q4", 0.0);
92     System* q5 = new System("Q5", 0.0);
93
94     //Criação e vinculação das 6 engrenagens de fluxos complexos
95     FlowComplex* f = new FlowComplex("f"); f->setSource(q1); f->setTarget(q2);
96     FlowComplex* g = new FlowComplex("g"); g->setSource(q1); g->setTarget(q3);
97     FlowComplex* r = new FlowComplex("r"); r->setSource(q2); r->setTarget(q5);
98     FlowComplex* t = new FlowComplex("t"); t->setSource(q2); t->setTarget(q3);
99     FlowComplex* u = new FlowComplex("u"); u->setSource(q3); u->setTarget(q4);
100     FlowComplex* v = new FlowComplex("v"); v->setSource(q4); v->setTarget(q1);
101
102     //Adicionando todos ao modelo por Agregação
103     Model* m = new Model("Complex Model Q");
104     m->add(q1); m->add(q2); m->add(q3); m->add(q4); m->add(q5);
105     m->add(f); m->add(g); m->add(r); m->add(t); m->add(u); m->add(v);
106
107     // m1 = m; (melhorar copia)
108
109     //Executando a simulação por 100 iterações
110     m->run(0, 100);
111
112     // teste (imprime qual a precisao do 77.1143 em 10 casas)
113     // std::cout << std::fixed << std::setprecision(10)
114     //     << q3->getValue() << " diff "
115     //     << std::fabs(q3->getValue() - 77.1143) << std::endl;
116
117
118     //Validação de alta precisao (tive que colocar a funcao round4 para que o codigo
pare de falhar quando muda a ultima casa decimal)
119     auto round4 = [](double v) {
120         return std::round(v * 10000.0) / 10000.0;
```

```

121     };
122
123     //verificacao do novo valor
124     // std::cout << std::fixed << std::setprecision(10)
125     //      << round4(q3->getValue());
126
127     assert(round4(q1->getValue()) == 31.8513);
128     assert(round4(q2->getValue()) == 18.4003);
129     assert(round4(q3->getValue()) == 77.1143);
130     assert(round4(q4->getValue()) == 56.1728);
131     assert(round4(q5->getValue()) == 16.4612);
132
133
134
135
136     std::cout << "Teste Complexo Passou!" << std::endl;
137
138     //Limpeza manual da memóriaHeap para evitar vazamentos (Memory Leak)
139     delete m;
140     delete q1; delete q2; delete q3; delete q4; delete q5;
141     delete f; delete g; delete r; delete t; delete u; delete v;
142 }

```

void exponentialFuncionalTest ()

```

10         { //Funcao para realizar os testes exponenciais
11     std::cout << "Executando Teste Exponencial..." << std::endl;
12
13     //Crio o modelo
14     Model* mod = new Model("Modelo Exponencial");
15     //Crio os sistemas
16     System* pop1 = new System("pop1", 100.0);
17     System* pop2 = new System("pop2", 0.0);
18     //Crio o fluxo
19     ExponencialFlow* exp = new ExponencialFlow("Fluxo Exponencial");
20
21     //Defino o destino e origem
22     exp->setSource(pop1); //origem
23     exp->setTarget(pop2); //destino
24
25     //Adiciono os sistemas e fluxo no modelo
26     mod->add(pop1);
27     mod->add(pop2);
28     mod->add(exp);
29
30     //Executo a simulação
31     mod->run(0, 100);
32
33     //Validação com assert
34     //Faço com que a diferença seja menor que 0.0001 para dar certo
35     assert(std::fabs(pop1->getValue() - 36.6032) < 0.0001);
36     assert(std::fabs(pop2->getValue() - 63.3968) < 0.0001);
37
38     std::cout << "Teste Exponencial Passou!" << std::endl;
39
40     //Limpo os ponteiros
41     delete mod;
42     delete pop1;
43     delete pop2;
44     delete exp;
45 }

```

void logisticalFuncionalTest ()

```
48         { //Funcao para realizar os testes logísticos
49     std::cout << "Executando Teste Logístico..." << std::endl;
50
51     //Crio o modelo e os sistemas
52     Model* mod = new Model("Modelo Logístico");
53     System* p1 = new System("p1", 100.0);
54     System* p2 = new System("p2", 10.0);
55     //Crio o fluxo
56     LogisticaFlow* log = new LogisticaFlow("Fluxo Logístico");
57
58     //Defino o destino e origem
59     log->setSource(p1); //origem
60     log->setTarget(p2); //destino
61
62     //Adiciono sistemas e fluxo no modelo
63     mod->add(p1);
64     mod->add(p2);
65     mod->add(log);
66
67     //Executo a simulacao
68     mod->run(0, 100);
69
70     //Validação com assert
71     //Faço com que a diferença seja menor que 0.0001 para dar certo
72     assert(std::fabs(p1->getValue() - 88.2167) < 0.0001);
73     assert(std::fabs(p2->getValue() - 21.7833) < 0.0001);
74
75     std::cout << "Teste Logístico Passou!" << std::endl;
76
77     //Deleto os ponteiros
78     delete mod;
79     delete p1;
80     delete p2;
81     delete log;
82 }
```

Referência do Arquivo test/funcional/funcional_tests.h

```
#include "../src/flow.h"
#include "../src/system.h"
```

Componentes

- class **ExponencialFlow** class **LogisticaFlow**
- class **FlowComplex**

Funções

- void **exponentialFuncionalTest** ()
- void **logisticalFuncionalTest** ()
- void **complexFuncionalTest** ()

Funções

void complexFuncionalTest ()

```
84         { //Funcao para realizar testes complexos
85     std::cout << "Executando Teste Complexo (Sistema Q)..." << std::endl;
86
87     //Criação dos 5 sistemas soltos na memória
88     System* q1 = new System("Q1", 100.0);
89     System* q2 = new System("Q2", 0.0);
90     System* q3 = new System("Q3", 100.0);
91     System* q4 = new System("Q4", 0.0);
92     System* q5 = new System("Q5", 0.0);
93
94     //Criação e vinculação das 6 engrenagens de fluxos complexos
95     FlowComplex* f = new FlowComplex("f"); f->setSource(q1); f->setTarget(q2);
96     FlowComplex* g = new FlowComplex("g"); g->setSource(q1); g->setTarget(q3);
97     FlowComplex* r = new FlowComplex("r"); r->setSource(q2); r->setTarget(q5);
98     FlowComplex* t = new FlowComplex("t"); t->setSource(q2); t->setTarget(q3);
99     FlowComplex* u = new FlowComplex("u"); u->setSource(q3); u->setTarget(q4);
100     FlowComplex* v = new FlowComplex("v"); v->setSource(q4); v->setTarget(q1);
101
102     //Adicionando todos ao modelo por Agregação
103     Model* m = new Model("Complex Model Q");
104     m->add(q1); m->add(q2); m->add(q3); m->add(q4); m->add(q5);
105     m->add(f); m->add(g); m->add(r); m->add(t); m->add(u); m->add(v);
106
107     // m1 = m; (melhorar copia)
108
109     //Executando a simulação por 100 iterações
110     m->run(0, 100);
111
112     // teste (imprime qual a precisao do 77.1143 em 10 casas)
113     // std::cout << std::fixed << std::setprecision(10)
114     //     << q3->getValue() << " diff "
115     //     << std::fabs(q3->getValue() - 77.1143) << std::endl;
116
117
118     //Validação de alta precisao (tive que colocar a funcao round4 para que o codigo
pare de falhar quando muda a ultima casa decimal)
119     auto round4 = [](double v) {
120         return std::round(v * 10000.0) / 10000.0;
121     };
122
```

```

123     //verificacao do novo valor
124     // std::cout << std::fixed << std::setprecision(10)
125     //     << round4(q3->getValue());
126
127     assert(round4(q1->getValue()) == 31.8513);
128     assert(round4(q2->getValue()) == 18.4003);
129     assert(round4(q3->getValue()) == 77.1143);
130     assert(round4(q4->getValue()) == 56.1728);
131     assert(round4(q5->getValue()) == 16.4612);
132
133
134
135
136     std::cout << "Teste Complexo Passou!" << std::endl;
137
138     //Limpeza manual da memóriaHeap para evitar vazamentos (Memory Leak)
139     delete m;
140     delete q1; delete q2; delete q3; delete q4; delete q5;
141     delete f; delete g; delete r; delete t; delete u; delete v;
142 }

```

void exponentialFuncionalTest ()

```

10                                     { //Funcao para realizar os testes exponenciais
11     std::cout << "Executando Teste Exponencial..." << std::endl;
12
13     //Crio o modelo
14     Model* mod = new Model("Modelo Exponencial");
15     //Crio os sistemas
16     System* pop1 = new System("pop1", 100.0);
17     System* pop2 = new System("pop2", 0.0);
18     //Crio o fluxo
19     ExponencialFlow* exp = new ExponencialFlow("Fluxo Exponencial");
20
21     //Defino o destino e origem
22     exp->setSource(pop1); //origem
23     exp->setTarget(pop2); //destino
24
25     //Adiciono os sistemas e fluxo no modelo
26     mod->add(pop1);
27     mod->add(pop2);
28     mod->add(exp);
29
30     //Executo a simulação
31     mod->run(0, 100);
32
33     //Validação com assert
34     //Faço com que a diferença seja menor que 0.0001 para dar certo
35     assert(std::fabs(pop1->getValue() - 36.6032) < 0.0001);
36     assert(std::fabs(pop2->getValue() - 63.3968) < 0.0001);
37
38     std::cout << "Teste Exponencial Passou!" << std::endl;
39
40     //Limpo os ponteiros
41     delete mod;
42     delete pop1;
43     delete pop2;
44     delete exp;
45 }

```

void logisticalFuncionalTest ()

```

48                                     { //Funcao para realizar os testes logísticos

```

```

49     std::cout << "Executando Teste Logistico..." << std::endl;
50
51     //Crio o modelo e os sistemas
52     Model* mod = new Model("Modelo Logistico");
53     System* p1 = new System("p1", 100.0);
54     System* p2 = new System("p2", 10.0);
55     //Crio o fluxo
56     LogisticaFlow* log = new LogisticaFlow("Fluxo Logistico");
57
58     //Defino o destino e origem
59     log->setSource(p1); //origem
60     log->setTarget(p2); //destino
61
62     //Adiciono sistemas e fluxo no modelo
63     mod->add(p1);
64     mod->add(p2);
65     mod->add(log);
66
67     //Executo a simulacao
68     mod->run(0, 100);
69
70     //Validação com assert
71     //Faço com que a diferença seja menor que 0.0001 para dar certo
72     assert(std::fabs(p1->getValue() - 88.2167) < 0.0001);
73     assert(std::fabs(p2->getValue() - 21.7833) < 0.0001);
74
75     std::cout << "Teste Logistico Passou!" << std::endl;
76
77     //Deleto os ponteiros
78     delete mod;
79     delete p1;
80     delete p2;
81     delete log;
82 }

```

funcional_tests.h

Ir para a documentação desse arquivo.

```
1 #ifndef FUNCIONAL_TESTS_H
2 #define FUNCIONAL_TESTS_H
3
4 #include "../../src/flow.h"
5 #include "../../src/system.h"
6
7 //Assinatura das funcoes
8 void exponentialFuncionalTest();
9 void logisticalFuncionalTest();
10 void complexFuncionalTest();
11
12 //Herança, exponencial herdando de flow
13 class ExponencialFlow : public Flow {
14 public:
15     //Herdando os construtores de forma canonica do flow
16     using Flow::Flow;
17
18     //Sobreescrita do metodo executa
19     double execute() const override {
20         //Pego a origem se for diferente de nulo
21         if (getSource() != nullptr) {
22             //Pego o valor atual e multiplico do 0.01
23             return 0.01 * getSource()->getValue();
24         }
25         return 0.0;
26     }
27 };
28
29 //Herança, logistica herdando de flow
30 class LogisticaFlow : public Flow {
31 public:
32     //Herdando os construtores de forma canonica do flow
33     using Flow::Flow;
34
35     //Sobreescrita do metodo executa
36     double execute() const override {
37         //Pego a origem se ela nao for nula
38         if (getTarget() != nullptr) {
39             //Pego o valor atual
40             double p2 = getTarget()->getValue();
41             //Faço as contas pedidas pelo cliente
42             return 0.01 * p2 * (1.0 - (p2 / 70.0));
43         }
44         return 0.0;
45     }
46 };
47
48 class FlowComplex : public Flow {
49 public:
50     //Herdando os construtores de forma canonica do flow
51     using Flow::Flow;
52
53     //Sobreescrita do metodo executa
54     double execute() const override {
55         if (getSource() != nullptr) {
56             return 0.01 * getSource()->getValue(); // Equação base do fluxo complexo
57         }
58         return 0.0;
59     }
60 };
61
```

```
62 #endif
```

Referência do Arquivo src/main.cpp

Referência do Arquivo test/funcional/main.cpp

```
#include "funcional_tests.h"
#include "../src/model.h"
#include "../src/system.h"
#include "../src/flow.h"
```

Definições e Macros

- #define MAIN_FUNCIONAL_TESTS

Funções

- int main ()

Definições e macros

```
#define MAIN_FUNCIONAL_TESTS
```

Funções

int main ()

```
9      {
10      //Roda os testes funcionais
11      exponentialFuncionalTest();
12      logisticalFuncionalTest();
13      complexFuncionalTest();
14      return 0;
15 }
```

Referência do Arquivo test/unit/main.cpp

Referência do Arquivo test/unit/unit_tests.cpp

Referência do Arquivo test/unit/unit_tests.h

unit_tests.h

Ir para a documentação desse arquivo.

1

Sumário

INDEX